

Username Enumeration via Different Responses

Name: Matyas Abreham

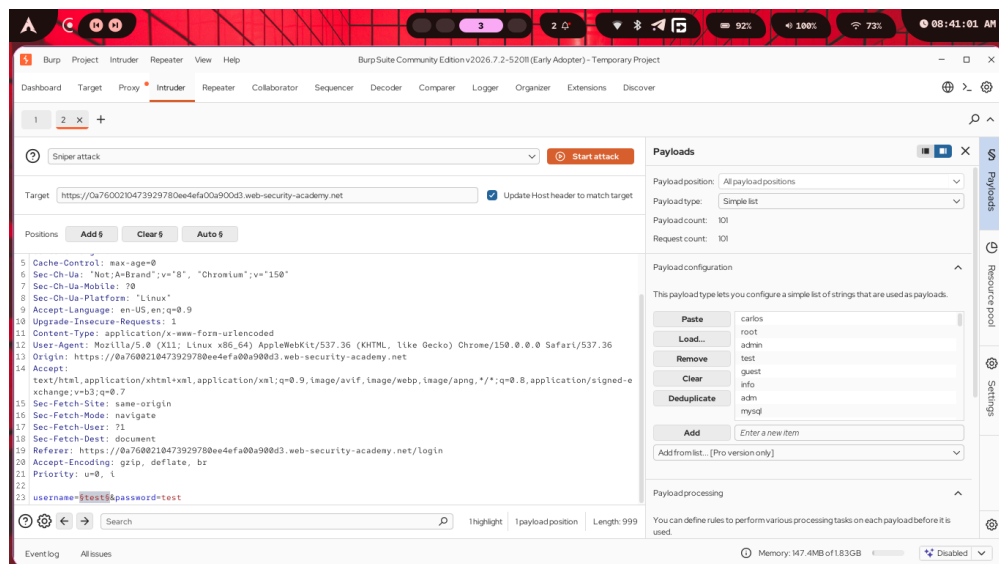
IDNO: CTC-207-26

Reconnaissance / Enumeration

This lab supplies two wordlists to work with - a list of candidate usernames and a list of candidate passwords - and the goal is to find the one real account hiding among them. I captured a normal login POST request in Burp and sent it to Intruder to automate the guessing instead of doing it by hand.

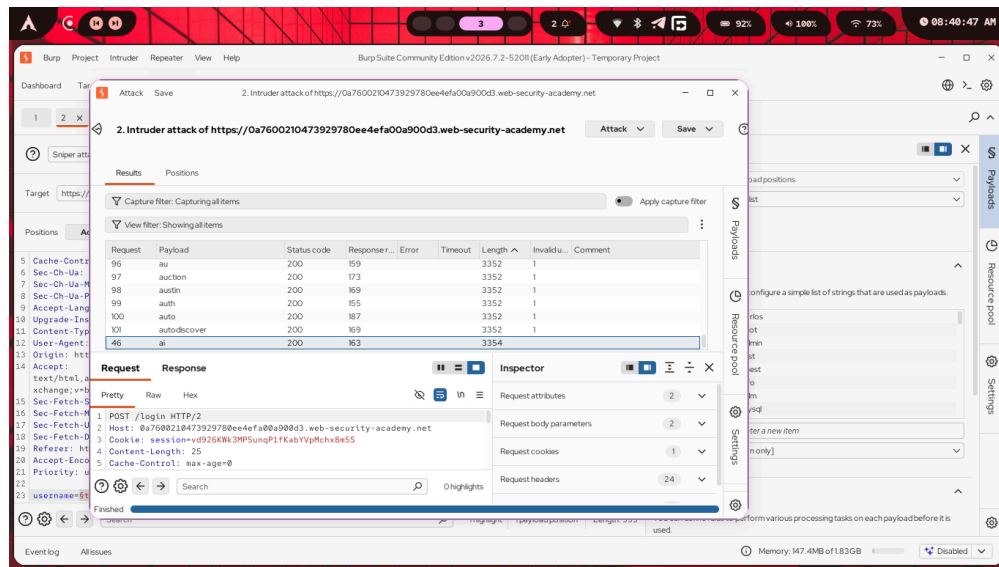


First pass: I set the attack position on the username parameter, kept the password fixed at a throwaway value ("test"), and loaded the full candidate username list as the payload for a Sniper attack.

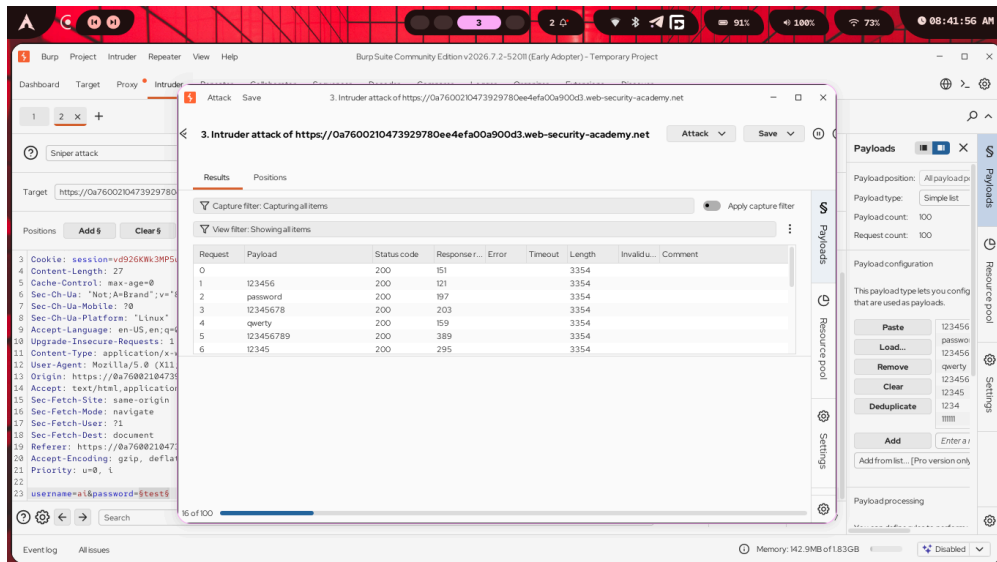


Vulnerability Discovered

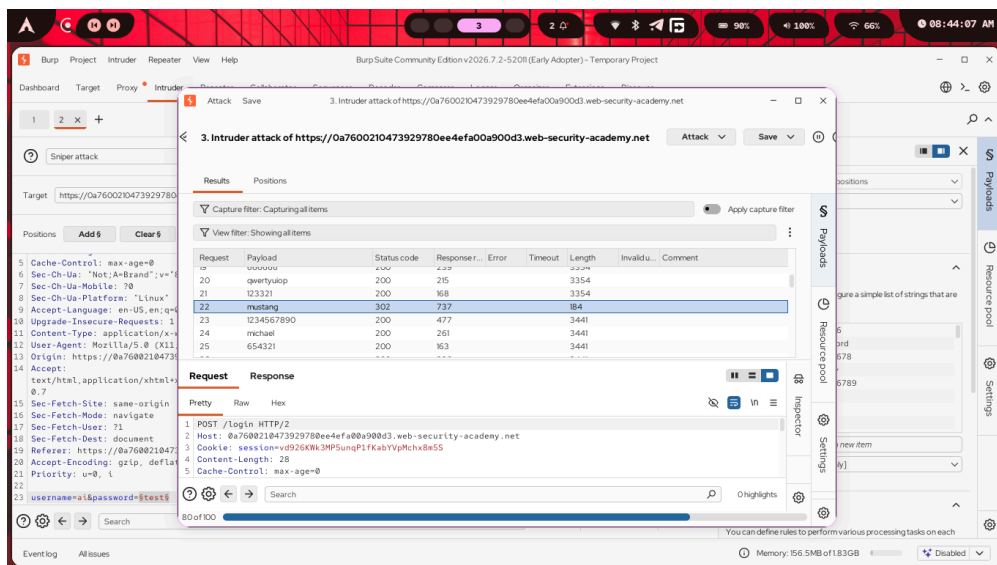
Running that attack and sorting by response length, almost every username returned an identical 3,352-byte response - the app's generic "invalid username or password" page. One entry broke that pattern: the candidate username ai came back at 3,354 bytes instead, just 2 bytes different. That's a subtle difference, but a real and consistent one, and it's exactly what this lab is built around - the app responds slightly differently depending on whether the username actually exists, even though the visible error message looks the same at a glance.



With a confirmed valid username in hand, I reused the exact same technique on the password field instead - fixed the username to ai, loaded the candidate password list onto the password parameter, and ran a second Snippet attack. There was no lockout, rate limiting, or CAPTCHA anywhere in either attack, so all 100 password attempts went through back-to-back without the app pushing back at all.



Almost every password attempt came back as a 200 with a near-identical response length (around 3,354-3,441 bytes) - the normal "login failed" page. One request broke the pattern completely: a 302 redirect with a response length of only 184 bytes, which is the app doing a successful post-login redirect instead of re-rendering the login form.



That anomalous request used the password `mustang`, giving the working credential pair `ai:mustang`. This is really two separate weaknesses stacked together: username enumeration via different responses (CWE-203) - the app leaks whether an account exists through inconsistent response length/content - combined with no protection at all against automated password brute-forcing once a valid username is known.

Exposed Information & Security Impact

The end result is a fully working login: username ai, password mustang. Logging in with those credentials gives complete access to that account, which on a real system could mean anything from reading someone's private data to full account takeover, depending on what that account can do.

The bigger issue is the pattern, not just this one account: because the app's response gives away whether ANY given username is valid, an attacker can enumerate a company's entire user base first, before even touching passwords. That valid-username list is valuable on its own for phishing campaigns and credential-stuffing attempts using passwords leaked from other breaches - and it turns what should be one hard two-unknowns guessing problem into two much easier one-at-a-time problems.

Remediation Recommendations

1. Make login failure responses byte-for-byte identical no matter what actually went wrong - same wording (a single generic "invalid username or password"), same HTTP status code, same response length, and ideally similar response timing, so there's no observable difference (not even 2 bytes) between "that username doesn't exist" and "that username exists but the password was wrong."
2. Add rate limiting and account lockout (or a CAPTCHA after a handful of failed attempts) on the login endpoint, tied to both the source IP and the target account. Even if usernames somehow do leak, this stops an attacker from being able to blast through an entire password wordlist unchallenged the way Intruder did here.
3. Enforce a reasonably strong password policy and check new passwords against known breached-password lists, so that even a determined, unthrottled brute-force attempt has a much lower chance of landing on something as guessable as a plain wordlist word.

Proof of Solve

Screenshot below shows the lab marked as Solved:

